Arrays and method arguments

CSC103 Programming Fundamentals / Lecture 21

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Write a method returning a new doubled array while leaving the supplied array unchanged. Test {2,3}.

Allocate an output array with input.length elements.

Learning objectives

- Pass and return arrays
- Distinguish mutation from parameter reassignment
- Use varargs and command-line arguments

CLO-2
Passing an array reference
Reassignment and returned arrays
Variable-length argument lists
Command-line arguments

Passing an array reference

- A parameter receives a copy of the array reference.
- Both references can reach the same array object.
- Changing an element is visible to the caller.

Example

```
static void setFirst(int[] a) {
  a[0] = 99;
}
```

Reassignment and returned arrays

- Reassigning the parameter does not reassign the caller reference.
- A method can construct and return a new array.
- A copy can protect the original data from later mutations.

Example

```
static int[] pair(int a, int b) {
  return new int[]{a, b};
}
```

Variable-length argument lists

- int... values accepts zero or more int arguments.
- Inside the method, values behaves as an array.
- A varargs parameter must be last.

Example

```
static int sum(int... values) {
  int total = 0;
  for (int v : values) total += v;
  return total;
}
```

Command-line arguments

- main receives command-line tokens as String elements.
- Check args.length before indexing.
- Parse a numeric argument before arithmetic.

Example

```
java Program 10 20

args[0] is "10"
args[1] is "20"
```

A new output array preserves the input

- An output array can hold transformed values while the caller keeps the original.
- The method allocates storage of the same length and writes each result by index.
- The returned reference identifies the new array.

Example

```
Input array: {2,3}
Output array: {4,6}
The two arrays have different identities.
```

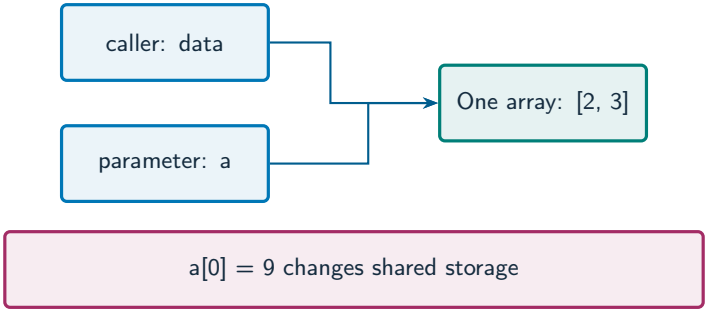
Reference copying does not copy elements

- Assigning int[] b=a makes b reach the same array.
- Creating new int[a.length] makes separate element storage.
- The choice determines whether later element updates are shared.

Example

```
int[] alias = a;       // same array
int[] copy = a.clone(); // independent primitive elements
```

A parameter copies an array reference



What to notice

Both references reach one array, so an element update is visible to the caller.

Key distinctions

Operation	New array?	Caller sees element changes?
b = a	No	Yes, shared array
a[0] = 9	No	Yes, element mutation
parameter = new int[2]	Yes	Caller variable unchanged
return new int[2]	Yes	Caller can store returned reference

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static void modify(int[] a) {
  a[0] = 9;
  a = new int[]{7};
}
static int sum(int... values) {
  int total = 0;
  for (int v : values) total += v;
  return total;
}
```

First example: execution

```
int[] data = {1, 2};
modify(data);
System.out.println(data[0]);
System.out.println(sum(2, 3, 4));
```

First example: result

9
9
modify changes the shared first element.
Its later parameter reassignment leaves the caller reference unchanged.

Variable-length argument lists
Command-line arguments

Guided practice

Write a method returning a new doubled array while leaving the supplied array unchanged. Test {2,3}.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Allocate an output array with input.length elements.
- Calculate each doubled value into the corresponding output position.
- Return the output and show that the original remains unchanged.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/2)

```
public class Solution21 {
  public static void main(String[] args) throws Exception {
    int[] original = {2, 3};
    int[] result = doubled(original);
    System.out.println(java.util.Arrays.toString(result));
    System.out.println(java.util.Arrays.toString(original));
  }
  static int[] doubled(int[] input) {
```

Complete solution (2/2)

```
    int[] output = new int[input.length];
    for (int i = 0; i < input.length; i++) {
      output[i] = input[i] * 2;
    }
    return output;
  }
}
```

Execution trace

Index	Input value	Output calculation	Output value
0	2	2 * 2	4
1	3	3 * 2	6
After return	Input still {2,3}	New array reference	{4,6}

Follow the changing state in execution order.

Solution result

[4, 6]
[2, 3]

Why this result follows
Return the output and show that the original remains unchanged.

A common incorrect approach

static void replace(int[] a) { a = new int[]{7}; }
// The caller expects its variable to become {7}.

Example

```
Return the new array and assign the result in the caller, or intentionally mutate the existing
    array elements.
```

Boundary and failure checks

Check the stated input assumptions.
Create a program that sums integer command-line arguments. Handle zero arguments explicitly.

Create an output array of the same length. Set out[i]=input[i]*2 and return out. Output: {4,6}. Input stays {2,3}.

Check your understanding

Does passing an array copy all its elements? Where must a varargs parameter occur?

Write a prediction and explain the rule that supports it.

Answer and explanation

No, it copies the reference value. The varargs parameter must be last.

Return the new array and assign the result in the caller, or intentionally mutate the existing array elements.

Compare cases and predict outcomes

Inside the method	Caller observes	Reason
a[0] = 9	Element becomes 9	Shared array
a = new int[]{7}	Original array unchanged	Only parameter reassigned
return a	Caller may store it	Explicit returned reference

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write a method that changes the first element to 9, then reassigns its parameter to a new array. Predict the caller array {2,3} afterward.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
static void change(int[] a) {
  a[0] = 9;
  a = new int[]{7};
}
// After change(data), data contains {9, 3}.
```

- The first statement mutates the original array.
- The second statement changes only the local parameter reference.
- The caller still reaches the original two-element array.

Copying a slice into another array

- Array assignment copies a reference; arraycopy copies elements between arrays.
- Specify source position, target position and the number of elements.
- For primitive elements the copied values are independent; reference elements would still be references.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: Copying a slice into another array

Task

Copy elements 20 and 30 from {10,20,30,40} into positions 0 and 1 of a new length-3 int array.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture21 {
  public static void main(String[] args) throws Exception {
    int[] source = {10, 20, 30, 40};
    int[] target = new int[3];
    System.arraycopy(source, 1, target, 0, 2);
    source[1] = 99;
    System.out.println(java.util.Arrays.toString(target));
  }

}
```

Worked trace and expected result

Copy step	Source element	Target position
First	source[1] = 20	target[0]
Second	source[2] = 30	target[1]
Not copied	Default zero	target[2]

Expected console output

```
[20, 30, 0]
```

Extend the ISB example

Practice

Why does the later source[1]=99 not change target[0]?

Explain your reasoning

The primitive int value was copied into separate storage. The two arrays are distinct objects.

Lecture summary

- and return arrays
- mutation from parameter reassignment
- varargs and command-line arguments

- Allocate an output array with input.length elements.
- Calculate each doubled value into the corresponding output position.
- Return the output and show that the original remains unchanged.

After-class practice

Create a program that sums integer command-line arguments. Handle zero arguments explicitly.

Syllabus reading
Deitel Ch 7
Next lecture: Two-dimensional arrays